

Integration Methods in cppDE

Simon Beyer

2026-09-10

Contents

1	Overview	2
2	cppODE()	2
2.1	Initial value problem and four methods	2
2.2	Stiffness and the choice of method	3
2.3	Multistep methods	3
2.4	Single-step methods	4
2.5	Adaptive step-size control	6
2.6	Dense output	7
2.7	Automatic Forward sensitivities	7
2.8	Events and restarts	9
2.9	Reverse-mode sensitivities: the discrete adjoint	13
2.10	Method selection	15
3	cnode()	16
3.1	What is shared	16
3.2	What differs	16
4	cppFUN()	17
4.1	Unified API and two computational paths	17
4.2	Chain rule via dX, dP, dX2, dP2	18
4.3	Pass-through of unmodelled inputs	18
5	Benchmarks	19
	References	21
A	BDF and NDF coefficients	22
B	Adams-Moulton coefficients	22
C	Rosenbrock4 coefficients	22
C.1	Diagonal coefficient, time partials, and node positions	23
C.2	Stage couplings α_{ij}	23
C.3	Stage couplings γ_{ij}	23
C.4	Solution and error weights	23
C.5	Dense-output coefficients	24
D	Tsitouras 5(4) coefficients	24

1 Overview

cppDE generates and compiles per-model C++ code for three related problems through three entry points that share a common R interface: `cppODE()`, `cvode()`, and `cppFUN()`. Table 1 summarises their scope.

	<code>cppODE()</code>	<code>cvode()</code>	<code>cppFUN()</code>
Problem class	ODE	ODE	algebraic $y = g(x, p)$
Methods	<code>bdf</code> , <code>adams</code> , <code>rb4</code> , <code>tsit5</code>	<code>bdf</code> , <code>adams</code>	
First-order sensitivities	in-tree dual-number AD	SUNDIALS analytic RHS	dual AD or symbolic
Reverse-mode gradient	yes (discrete adjoint)	no	yes
Second-order sensitivities	yes (nested duals)	no	yes (nested duals or symbolic)
External dependencies	LAPACK; KLU optional	SUNDIALS ≥ 6.0 ; KLU opt.	none
Codegen	SymPy + R CMD SHLIB	SymPy + R CMD SHLIB	SymPy + R CMD SHLIB

Table 1: The three cppDE entry points and what they cover. The codegen pipeline (parse the user’s expressions through SymPy, emit C++, compile via `R CMD SHLIB`, load via `dyn.load`) is shared, as is the `solveODE()` dispatcher used by both integrator backends.

The integrator entry points `cppODE()` and `cvode()` expose the same R-side interface: the user supplies named character vectors of right-hand sides together with optional events, forcings, and parameter vectors, and the returned model handle is passed to `solveODE()`. The two backends differ only in the numerical core. `cppODE()` emits a self-contained C++ source that includes the headers under `inst/include/cppde/`; the resulting binary depends on LAPACK and, optionally, KLU. `cvode()` emits a thin glue layer that links against the system-installed SUNDIALS CVODE/CVODES library and inherits its numerical behaviour.

Section 2 describes the in-tree integrator `cppODE()` in full, including its four time-stepping methods, its forward-mode automatic-differentiation framework for parameter sensitivities, the reverse-mode discrete adjoint that computes a gradient at a cost independent of the number of parameters, and the transport of sensitivities across events. Section 3 describes the SUNDIALS-backed `cvode()` integrator. Section 4 describes the algebraic-function compiler `cppFUN()`, which compiles a system of expressions $y = g(x, p)$ together with optional first- and second-order derivatives, with no time integration. The principal use cases of `cppFUN()` are observation maps that link integrator output to data in likelihood-based inference and reparametrisation Jacobians $\partial p / \partial \theta$ that seed sensitivity propagation through the integrators. Section 5 benchmarks the two integrator backends against each other on 42 problems.

2 cppODE()

2.1 Initial value problem and four methods

This section describes the in-tree integrator of `cppODE()` in detail: the four time-stepping methods, the adaptive step-size control they share, dense Hermite output, sparse and dense LU factorisation, time- and root-triggered events, and forward-mode sensitivity propagation through dual numbers. The methods differ only in how the individual time step is taken. The Butcher tableaus and numerical coefficients of each method are reproduced in Appendices A–D.

The initial-value problem solved by `cppODE()` is

$$\dot{x} = f(x, p_{\text{dyn}}, t), \quad x(t_0) = p_{\text{init}}, \quad (1)$$

with state $x \in \mathbb{R}^{n_x}$ and parameter vector $p = (p_{\text{init}}, p_{\text{dyn}})$ collecting the initial conditions and the dynamic parameters. The right-hand side f may depend explicitly on time, in which case the system is non-autonomous; an explicit time argument is also necessary for the forcing-function machinery used to inject time-dependent inputs. The Jacobian of f with respect to the state is denoted $J = \partial f / \partial x$ throughout.

2.2 Stiffness and the choice of method

The choice between the four methods is governed primarily by whether the problem is *stiff*. A problem is stiff on a given integration interval when its solution contains components evolving on time scales much shorter than the interval length. An explicit integration method applied to such a problem is forced to take step sizes set by the fastest component rather than by the accuracy requirement, leading to step counts that are uneconomic in practice. Implicit methods avoid this constraint by absorbing the fast components into a linear solve at each step.

The standard tool for analysing the stability of a method is the linear scalar test equation $\dot{y} = \lambda y$ with $\lambda \in \mathbb{C}^-$, whose exact solution decays. Substituting this equation into a one-step method produces a single scalar amplification factor $R(z)$ acting on y per step, where $z = h\lambda$ combines the step size and the eigenvalue. The method is *A-stable* if $|R(z)| \leq 1$ on the entire left half-plane, so that the numerical solution decays whenever the exact one does, regardless of step size; *A(α)-stable* if the same bound holds on the wedge $\{z \in \mathbb{C} : |\arg(-z)| \leq \alpha\}$ for some opening angle $\alpha < 90^\circ$; and *L-stable* if it is A-stable and additionally $R(z) \rightarrow 0$ as $\text{Re}(z) \rightarrow -\infty$, the latter condition being what damps very stiff transients out in a single step rather than carrying them along.

The initial-value problem (1) is called *stiff* on $[t_0, t_{\text{end}}]$ when an explicit method is forced to take step sizes much smaller than what local accuracy alone would demand, in order to keep the dimensionless products $h\lambda_k$ inside the stability domain, where $\{\lambda_k\}$ are the eigenvalues of J along the trajectory. Stiffness is therefore not a property of the system in isolation; it depends on the spectrum of J , on the integration horizon, and on the requested tolerance. Chemical-kinetics networks with rate constants spanning several orders of magnitude are stiff in this sense, as are many discretised parabolic PDEs.

Stiff problems require methods whose stability domain contains the entire left half-plane (or a wide wedge thereof). Of the four methods listed above, BDF/NDF and Rosenbrock4 satisfy this requirement; Adams-Moulton and Tsit5 do not, and will perform poorly on stiff systems regardless of the requested tolerance.

2.3 Multistep methods

Linear multistep methods build the solution at the new time step x_n from a linear combination of the most recent $k + 1$ states and right-hand-side values, $\{x_{n-j}, f_{n-j}\}_{j=0}^k$. They re-use information from earlier steps and so attain high accuracy with few right-hand-side evaluations per step, at the price of having to manage that history.

cppDE implements both the BDF and the Adams families through a common internal representation, due to Nordsieck and used throughout SUNDIALS [1]. The idea is to store, instead of the past $k + 1$ states, a single *scaled derivative array*

$$z_n = \left(x_n, h_n \dot{x}_n, \frac{h_n^2}{2!} \ddot{x}_n, \dots, \frac{h_n^k}{k!} x_n^{(k)} \right),$$

i.e. the value at the current time and its first k derivatives, each multiplied by the appropriate power of the current step size to make the entries dimensionally homogeneous. Changing the step size or the integration order then becomes a single linear transformation of z_n rather than a refit through the past data, which simplifies both the bookkeeping and the implementation of variable-order and variable-step control.

Backward Differentiation Formula (bdf)

The BDF formula of order q defines x_n implicitly by the condition

$$\sum_{j=0}^q \alpha_j x_{n-j} = h_n f(x_n, t_n), \quad (2)$$

which is exact whenever $x(t)$ is a polynomial of degree at most q on the past $q + 1$ nodes. The method is A-stable for $q \leq 2$ and A(α)-stable for $q \in \{3, 4, 5\}$; the stability angle α shrinks monotonically from 86° at $q = 3$ to 51° at $q = 5$.

The implicit corrector (2) is solved by Newton iteration,

$$W \Delta x^{(\nu)} = -\tilde{r}(x_n^{(\nu)}), \quad x_n^{(\nu+1)} = x_n^{(\nu)} + \Delta x^{(\nu)}, \quad (3)$$

with leading coefficient $\beta_0 = 1/\alpha_0$ and the iteration matrix

$$W = \frac{1}{h_n \beta_0} \mathbb{1} - J(x_n^{(\nu)}, t_n), \quad (4)$$

in the form adopted by SUNDIALS [1], which differs from the textbook expression $I - h_n \beta_0 J$ by the overall scalar factor $h_n \beta_0$. The factorisation of W is the dominant cost per accepted step and is reused for as long as the contraction rate of the Newton iteration permits.

The default in cppDE is the Numerical Differentiation Formula (NDF) variant of [2], which adds a correction proportional to the Nordsieck predictor error to the BDF corrector,

$$\sum_{m=1}^q \frac{1}{m} \nabla^m x_n - h_n f(x_n, t_n) - \kappa_q \gamma_q (x_n - x_n^{\text{pred}}) = 0, \quad \gamma_q = \sum_{m=1}^q \frac{1}{m}, \quad (5)$$

where ∇ denotes the backward-difference operator. The algebraic effect of the modification is to scale β_0 in (3) by a factor $(1 - \kappa_q)$ and to multiply the leading truncation-error coefficient $1/(q+1)$ by the factor $1 + \kappa_q(q+1)\gamma_q$. With $\kappa_q < 0$ the truncation error decreases at fixed step size while the iteration matrix is unchanged, which permits a larger step at a fixed accuracy requirement. The values of κ_q adopted by cppDE are tabulated in Appendix A; they reproduce the 25% step-size gain over classical BDF reported by [2] for $q \leq 4$, at the cost of a small reduction in stability angle. Setting `useNDF = FALSE` recovers the classical BDF formulas ($\kappa_q \equiv 0$).

A typical accepted step performs one right-hand-side evaluation for the predictor and between one and three right-hand-side evaluations together with the same number of linear solves for the corrector. A new Jacobian evaluation and LU factorisation are triggered only when the Newton iteration stalls or when the step size or order changes substantially. BDF/NDF is the default method in cppDE and the recommended choice when stiffness is suspected.

Adams-Moulton in PECE form (adams)

The Adams-Moulton family integrates the local interpolant of f over $[t_{n-1}, t_n]$. Writing f as the unique polynomial through the $q+1$ samples $\{(t_{n-j}, f_{n-j})\}_{j=0}^q$ and integrating gives

$$x_n = x_{n-1} + h_n \sum_{j=0}^q \beta_j f(x_{n-j}, t_{n-j}), \quad (6)$$

which is implicit in x_n through the $j=0$ term. cppDE evaluates the corrector in predict-evaluate-correct-evaluate (PECE) mode rather than by Newton iteration. An explicit predictor (Adams-Bashforth of the same order q) first supplies a candidate value x_n^P for the new state; the right-hand side is then evaluated at x_n^P ; the corrector (6) is evaluated explicitly with $f(x_n^P, t_n)$ in place of $f(x_n, t_n)$, yielding a refined x_n ; and a final right-hand-side evaluation at the refined state is entered into the Nordsieck history. The procedure costs two right-hand-side evaluations per accepted step, with no nonlinear iteration and no linear solve.

The Adams-Moulton stability domain on the negative real axis shrinks rapidly with order; for $q \geq 3$ it does not contain even a small wedge of $\mathbb{C}^-$ away from the origin, so the method is unsuitable for stiff problems. For non-stiff problems with smooth solutions and a long integration horizon, however, the high attainable order ($q_{\text{max}} = 12$) and the cost of two right-hand-side evaluations per accepted step make it competitive with explicit Runge-Kutta methods of fixed order.

2.4 Single-step methods

Single-step methods carry no information beyond the current state. They achieve order $p > 1$ by evaluating internal stages within each step rather than by reaching back into the integration history. The absence of history makes restarts after events trivial; both methods below restart at full order in a single step, unlike the multistep methods, which must rebuild the Nordsieck history at order 1.

Rosenbrock (rb4)

A Rosenbrock method of s stages with diagonal coefficient γ linearises the implicit Runge-Kutta condition about (x_n, t_n) and defines stage values k_i by

$$Wk_i = f \left(x_n + \sum_{j<i} \alpha_{ij} k_j, t_n + c_i h_n \right) + \frac{1}{h_n} \sum_{j<i} \frac{\gamma_{ij}}{\gamma} k_j + h_n d_i \frac{\partial f}{\partial t}(x_n, t_n), \quad (7)$$

with the iteration matrix

$$W = \frac{1}{\gamma h_n} \mathbb{1} - J(x_n, t_n). \quad (8)$$

The method coefficients are listed in Table 2.

Symbol	Meaning
α_{ij}	weights with which stage i combines all earlier stages k_j , $j < i$
c_i	time abscissa of stage i inside the step, with $c_i = \sum_{j<i} \alpha_{ij}$
γ_{ij}	weights with which the linear-solve correction at stage i uses earlier stages
γ	common diagonal coefficient appearing in the iteration matrix W
d_i	weights that pick up the explicit time dependence of f via $\partial f / \partial t$
m_i	weights combining the stages into the advanced solution x_{n+1}
$\hat{m}_i$	weights combining the stages into the embedded order-3 estimator used for step control

Table 2: Rosenbrock method coefficients.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + \sum_{i=1}^s m_i k_i, \quad \hat{x}_{n+1} = x_n + \sum_{i=1}^s \hat{m}_i k_i.$$

Each stage (7) is a single linear solve; no Newton iteration is required. Per accepted step `cppDE` evaluates J and factorises W once, performs five right-hand-side evaluations, and back-substitutes six times.

`cppDE` uses the order-4 coefficients of [3, Sec. IV.7], the same set as the `rosenbrock4` stepper of `Boost.NumericOdeint`. The pair is L-stable and carries an embedded order-3 estimator. Continuous output on $[t_n, t_{n+1}]$ is delivered by an order-3 interpolant constructed from the stage values k_i and two additional coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$ tabulated in Appendix C; no extra right-hand-side evaluations are required for output points.

Rosenbrock methods are competitive with BDF on stiff problems whose Jacobian is cheap relative to the right-hand side, and they avoid the order-rebuilding cost incurred by the multistep methods after an event. Rosenbrock4 is therefore the recommended stiff method for systems with frequent state discontinuities at moderate state dimension.

Tsitouras (tsit5)

Tsitouras is the explicit Runge-Kutta pair of order 5(4) derived by [4] under the simplifying assumption that the first column of the coefficient matrix vanishes, $a_{i,1} = 0$ for $i = 2, \dots, s$. Removing this column reduces the number of order conditions to be satisfied during construction and was shown to admit a particularly favourable choice of nodes c_i in terms of leading-error norm. The pair has $s = 7$ stages with the First-Same-As-Last (FSAL) property,

$$k_7 = f(x_{n+1}, t_n + h_n) = k_1 \text{ at step } n+1,$$

so an accepted step costs six right-hand-side evaluations rather than seven. The method coefficients are listed in Table 3.

Symbol	Meaning
a_{ij}	weights with which stage i combines earlier stages (lower triangular; first column is zero)
c_i	time abscissa of stage i inside the step, with $c_i = \sum_{j<i} a_{ij}$
b_i	weights combining the stages into the advanced order-5 solution; $b_7 = 0$
$\hat{b}_i$	weights combining the stages into the embedded order-4 estimator; $\hat{b}_7 \neq 0$
k_i	right-hand side at stage i : $k_i = f\left(x_n + h_n \sum_{j<i} a_{ij} k_j, t_n + c_i h_n\right)$

Table 3: Tsit5 method coefficients.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + h_n \sum_{i=1}^7 b_i k_i, \quad \hat{x}_{n+1} = x_n + h_n \sum_{i=1}^7 \hat{b}_i k_i,$$

and the local error estimate used for step-size control is

$$e_{n+1} = h_n \sum_{i=1}^7 (\hat{b}_i - b_i) k_i.$$

Continuous output uses the accepted-step endpoints and their derivatives $(f_n, f_{n+1}) = (k_1, k_7)$, both already in cache from the FSAL property, so an output point costs no further right-hand-side evaluation.

Tsitouras has no Jacobian and no linear solve and performs the largest amount of integration per right-hand-side call of the four methods considered here. It is the appropriate choice for non-stiff problems whose right-hand side is expensive to evaluate.

2.5 Adaptive step-size control

The error estimate e_n delivered by each method is reduced to a scalar through the weighted root-mean-square norm

$$\|e_n\|_{\text{WRMS}} = \sqrt{\frac{1}{n_x} \sum_{j=1}^{n_x} \left(\frac{e_{n,j}}{w_{n,j}} \right)^2}, \quad w_{n,j} = \text{atol} + \text{rtol} |x_{n,j}^{\text{ref}}|, \quad (9)$$

in which $w_{n,j}$ is a per-component tolerance combining an absolute floor **atol** and a relative scale **rtol** $|x_{n,j}^{\text{ref}}|$. Normalising $e_{n,j}$ by $w_{n,j}$ before squaring places components of widely differing magnitudes on a common scale and removes the dependence of the acceptance test on the physical units of the state. The reference state x_n^{ref} that enters the relative term is the Nordsieck predictor in BDF/NDF and the component-wise maximum $\max(|x_n|, |x_{n+1}|)$ in the single-step methods; the two choices are both standard and produce numerically indistinguishable values of the norm in practice. A step is accepted when $\|e_n\|_{\text{WRMS}} \leq 1$.

The next step size h_{n+1} is selected by a control law of the type analysed in [3], applied to the WRMS norm of the current step and, when active, the WRMS norm of the previous step,

$$h_{n+1} = h_n \min \left\{ \rho_{\max}, \max \left(\rho_{\min}, S \|e_n\|_{\text{WRMS}}^{-1/(p+1)} \|e_{n-1}\|_{\text{WRMS}}^{\beta/(p+1)} \right) \right\},$$

with safety factor $S < 1$, growth limits $\rho_{\min}, \rho_{\max}$, and a memory coefficient $\beta \in [0, 1]$. The case $\beta = 0$ is the standard integral or I-controller, in which h_{n+1} depends on the current error norm only; cppDE uses this form for the multistep methods, where the Nordsieck representation already smooths the underlying signal. The case $\beta > 0$ is the proportional-integral (PI) controller of [3]; the single-step methods adopt this form to damp the high-frequency oscillations in h_n that a pure I-controller can otherwise produce on stiff or near-singular problems.

The first step

The controller needs a step to correct, and the first one has no error norm behind it. cppDE estimates it from the initial state in the manner of [5], Algorithm II.4.4: a rough $h_0 \sim \|x_0\|/\|f_0\|$ fixes a scale, a second derivative $\ddot{x}$ along the trajectory fixes the curvature, and

$$h_1 = \left(\frac{0.01}{\max(\|f_0\|, \|\ddot{x}\|)} \right)^{1/(p+1)} \quad (10)$$

turns the two into a step for a method of order p . The implicit methods obtain $\ddot{x} = \partial f / \partial t + Jf$ from the generated Jacobian, the explicit one from a finite difference of f . All norms are the weighted norms of (9), so for an AD value type the derivative components enter as well and the sensitivity dynamics influence the first step.

Two bounds follow, in the form CVODE uses. An upper bound keeps a single Euler step from changing any component by more than a fraction of its own scale, and is capped by the remaining interval; a lower bound keeps h_0 above the rounding of the time variable itself. The multistep methods reach the same estimate through a port of the CVODES routine `cvHin`, which refines $\sqrt{h_{\text{lb}} h_{\text{ub}}}$ by a finite-difference curvature until two successive estimates agree within a factor of two, so that the first step cppDE takes is the one CVODES would take on the same problem. The same estimator runs again after every event, on the post-event state.

2.6 Dense output

User-requested output times do not in general coincide with the adaptive step grid produced by the controller. Each accepted step therefore carries a local interpolant that allows evaluation of $x(t)$ at any time inside that step at the cost of polynomial arithmetic only. cppDE uses Hermite interpolation, which constrains both values and derivatives of the interpolant at the step endpoints. This form attains the same order as the underlying step without recourse to data outside the step: state and right-hand side at the two endpoints, augmented by stage values for the single-step methods. No additional right-hand-side evaluations are required for dense output. The local interpolation order is 4 for Rosenbrock4 and Tsit5; the multistep methods reuse the Nordsieck history, which carries derivatives up to order q .

2.7 Automatic Forward sensitivities

Dual-number arithmetic

Parameter sensitivities of the trajectory are computed by forward-mode automatic differentiation rather than by finite differences (whose truncation and round-off errors require multiple integrations to control) or by symbolic derivation of the sensitivity equations (whose intermediate expressions grow rapidly with the dimension of f). Forward-mode AD augments every floating-point scalar in the computation with an additional component that records its directional derivative with respect to a chosen parameter direction. The arithmetic operations and elementary functions are overloaded to act on this augmented type so that the chain rule is applied automatically; the C++ source that evaluates f is then compiled unchanged for both the primal and the sensitivity computation.

The augmented type is the *dual number*

$$\tilde{x} = x + x' \varepsilon, \quad \varepsilon^2 = 0, \quad \varepsilon \neq 0,$$

where $x \in \mathbb{R}$ is the primal value and $x' \in \mathbb{R}$ is the directional derivative. Treating $\tilde{x}$ as the first two terms of a Taylor expansion in ε and discarding all higher powers (which vanish by $\varepsilon^2 = 0$), the arithmetic of dual numbers follows from the standard rules of calculus: for $\tilde{x} = x + x' \varepsilon$ and $\tilde{y} = y + y' \varepsilon$,

$$\tilde{x} + \tilde{y} = (x + y) + (x' + y') \varepsilon, \quad \tilde{x} \cdot \tilde{y} = xy + (xy' + x'y) \varepsilon,$$

and for any smooth scalar function g ,

$$g(\tilde{x}) = g(x) + g'(x) x' \varepsilon.$$

The chain rule is therefore intrinsic to the algebra: any composition of dual-number operations automatically tracks the derivative of the composed function, and no code transformation or symbolic manipulation is required. The same source that evaluates f over $\mathbb{R}$ evaluates both f and $\partial f/\partial x$ when called with dual-number inputs, by virtue of the overloaded operators and elementary functions.

Sensitivity equations via dual evaluation

Let the ODE right-hand side depend on a parameter vector $p \in \mathbb{R}^{n_p}$,

$$\dot{x} = f(x, p, t), \quad x(t_0) = x_0(p),$$

and define the *sensitivity matrix*

$$S_p = \frac{\partial x}{\partial p} \in \mathbb{R}^{n_x \times n_p}, \quad (S_p)_{ij} = \frac{\partial x_i}{\partial p_j}.$$

To derive the evolution equation for S_p via dual arithmetic, seed each state component with its row of S_p and form the dual-valued state vector

$$\tilde{x}_i = x_i + (S_p)_{ij} \varepsilon_j,$$

where ε_j is the dual unit associated with direction p_j and summation over j is implied. Evaluating f component-wise over the dual algebra and collecting the ε_j -coefficient of the i -th output gives

$$[f(\tilde{x}, p, t)]_{i, \varepsilon_j} = \frac{\partial f_i}{\partial x_k} (S_p)_{kj} + \frac{\partial f_i}{\partial p_j},$$

where repeated k is summed. Since the primal part of the dual evaluation recovers $\dot{x}_i = f_i(x, p, t)$, differentiating $\dot{x}_i = f_i$ with respect to p_j and identifying the ε_j -coefficient with $(\dot{S}_p)_{ij}$ yields

$$\dot{S}_p = \frac{\partial f}{\partial x} S_p + \frac{\partial f}{\partial p}, \quad S_p(t_0) = \frac{\partial x_0}{\partial p}. \quad (11)$$

Hence a single dual-number evaluation of f propagates the full sensitivity matrix S_p in lockstep with the primal state x , at a cost proportional to n_p . No additional Jacobian evaluations are required; the same code path that computes f computes $(\partial f/\partial x) S_p + \partial f/\partial p$ automatically.

Seeding: from S_p to S_θ

In many applications the natural parameters of inference are not the raw entries of p but a higher-level vector $\theta \in \mathbb{R}^M$ to which both the dynamic parameters p_{dyn} and the initial condition $x_0 = p_{\text{init}}(\theta)$ are linked through a user-defined reparametrisation $p = \Phi(\theta)$. The quantity of interest is then

$$S_\theta = \frac{\partial x}{\partial \theta} \in \mathbb{R}^{n_x \times M},$$

related to S_p by the chain rule $S_\theta = S_p \partial p/\partial \theta$. Substituting this relation into (11) yields the evolution equation for S_θ directly,

$$\dot{S}_\theta = \frac{\partial f}{\partial x} S_\theta + \frac{\partial f}{\partial p_{\text{dyn}}} \frac{\partial p}{\partial \theta}, \quad S_\theta(t_0) = \frac{\partial p_{\text{init}}}{\partial \theta}. \quad (12)$$

The matrix $\partial p/\partial \theta$ is the *seed matrix*, supplied by the user (through the `sens1ini` argument of `solveODE()`) and held fixed for the duration of the integration. The choice $\partial p/\partial \theta = I_{n_p}$ recovers the canonical case $S_\theta = S_p$; any other choice carries the sensitivity computation in M specified directions of θ -space at the same cost as integrating M additional states alongside x .

Dual-number aware error norm

When forward sensitivities are active, the integrator carries the sensitivity matrix $S_\theta \in \mathbb{R}^{n_x \times M}$ alongside the primal state. Every accepted step then produces $1 + M$ error vectors of length n_x : the primal estimate $e_n^{(x)} \in \mathbb{R}^{n_x}$ and a sensitivity estimate $e_S^{(k)} \in \mathbb{R}^{n_x}$ for each column $\partial x / \partial \theta_k$ of S_θ . Two natural ways to combine these into a single scalar suggest themselves: a flat WRMS that pools all $n_x(1 + M)$ components into one sum, or a maximum over per-vector WRMS norms. cppDE adopts the second, following the staggered-tolerance convention of CVODES (the `CV_STAGGERED` mode of its sensitivity solver).

The state norm $\|e_n^{(x)}\|_{\text{WRMS}}$ is exactly (9). The sensitivity column norm is its analogue, with S_θ replacing x in both the error vector and the weight,

$$\|e_S^{(k)}\|_{\text{WRMS}} = \sqrt{\frac{1}{n_x} \sum_{i=1}^{n_x} \left(\frac{(e_S)_{ik}}{w_i^{(k)}} \right)^2}, \quad w_i^{(k)} = \text{atol} + \text{rtol} |(S_\theta)_{ik}^{\text{ref}}|, \quad (13)$$

in which $(S_\theta)_{ik}^{\text{ref}}$ is the reference value of the i -th component of column k , taken from the same source (predictor or pre/post maximum) as x_n^{ref} for the state. The scalar that the step-size controller sees is the worst of the state norm and the M column norms,

$$\|e_n\|_{\text{WRMS}} = \max \left\{ \|e_n^{(x)}\|_{\text{WRMS}}, \max_{1 \leq k \leq M} \|e_S^{(k)}\|_{\text{WRMS}} \right\}. \quad (14)$$

The maximum-of-norms convention is preferred to the flat WRMS, which divides by the total component count $n_x(1 + M)$ and therefore averages the squared errors across directions in θ : a large error in one sensitivity column k_0 can be obscured by M small errors in the remaining columns and the state, leading the integrator to accept a step that is too large in the direction k_0 . The maximum norm prevents this dilution and forces each direction in θ to satisfy the requested tolerance independently. Because the controller still operates on a single scalar, the cost relative to the flat norm is a constant overhead in the number of accepted steps, and the control-law parameters $(S, \rho_{\min}, \rho_{\max}, \beta)$ require no adjustment.

The same WRMS structure governs the Newton convergence test in BDF/NDF. A Newton iterate is accepted when the iteration correction satisfies (14), with the maximum again taken over the state and the per-column sensitivity components. Convergence in the primal therefore does not allow a loose solve in any sensitivity direction; a parameter direction that drives the correction norm up triggers either further Newton iterations or a step-size reduction, even when the primal residual is already small.

2.8 Events and restarts

cppDE supports two kinds of events that interrupt smooth integration. A *time-triggered event* fires at a user-specified time $t_e(\theta)$ that may depend on θ through a closed-form expression. A *root-triggered event* fires when a user-supplied event function $g_e(x, t)$ crosses zero along the trajectory; the firing time $t^*(\theta)$ is then defined implicitly by $g_e(x(t^*), t^*) = 0$. In both cases an event applies a discontinuous reset to the state,

$$x^+ = g(x^-, t),$$

typical examples being instantaneous dosing of a kinetic species, a clipping rule that enforces non-negativity, or a switch between dynamical regimes. The remainder of this section first describes the cost of restarting the integrator after a discontinuity, then sets up the sensitivity-transport problem, and finally derives the correction for each event type.

Restart cost

Single-step methods (Rosenbrock4, Tsit5) carry no information beyond the current state and resume integration in a single full-order step after the reset. Multistep methods (BDF/NDF, Adams) build their order through history and must restart at order 1, rebuilding the Nordsieck array from scratch over several steps. This restart penalty is the dominant cost on problems with frequent events and is the reason Rosenbrock4 is the recommended stiff method when discontinuities are common, even though BDF/NDF is the default elsewhere.

The reset also moves the state onto a different trajectory, so the step size that suited the regime before the event is not a starting point for the one after it. cppDE re-estimates it for every method from the post-event state, with the same initial-step estimator the solve begins with and the remaining interval as its upper bound. For a single-step method that estimate is the whole restart; for a multistep method it comes on top of the order-1 restart above.

Sensitivity transport across a discontinuity

The reset map g acts on the trajectory $x(t, \theta)$ at the firing time $t^*(\theta)$. The naive update of the sensitivity, by the linearisation of g around the pre-event state, $S_\theta^+ = (\partial g / \partial x) S_\theta^-$, is incorrect in the presence of an event because the firing time itself depends on θ . A perturbation δt^* of the firing time induces short integrations of $\dot{x} = f$ on either side of the event, with f taking different values before and after the reset. Denote by f^- the right-hand side just before the reset, evaluated at x^- , and by f^+ the right-hand side just after the reset, evaluated at $g(x^-, t^*)$. The sensitivity at the first grid time following the event then receives two contributions in addition to the linearised reset:

1. *Pre-event drift.* The pre-event trajectory x^- flows along $\dot{x} = f^-$ from the grid time $t_*^{(0)} := \text{scalar}(t^*)$ to the θ -perturbed firing time t^* , over a displacement $\delta t^* = t^* - t_*^{(0)}$ whose scalar part vanishes but whose θ -derivatives do not.
2. *Post-event drift.* The reset state $g(x^-, t^*)$ then flows along $\dot{x} = f^+$ from t^* back to the grid time over the negative of the same displacement.

Both shifts contribute terms of order $f^\pm (\partial t^* / \partial \theta)$ to the sensitivity at the grid time. The correct first-order update reproduces a closed-form expression known as the *saltation matrix*, recovered below as a consequence of the construction.

cppDE realises this correction by replacing the bare reset with a forward-drift-reset-backward-drift sequence, evaluated in dual-number arithmetic. Each drift is integrated by Heun's method, a two-stage second-order Runge-Kutta scheme also known as the improved Euler method, which averages the right-hand side at the start and the end of the interval. Applied to dual-typed input the average is a pure arithmetic operation: the scalar component of x at the grid time is unchanged (since the displacement δt^* has scalar part zero), but the θ -derivatives of the displacement propagate into the state. A single Euler step would suffice for first-order sensitivities, but its truncation error of order δt^{*2} contributes a non-vanishing second-order θ -derivative through the dual algebra and would distort the Hessian. Heun's method has truncation error $O(\delta t^{*3})$ and recovers the correct second-order sensitivities. The reset map g is applied between the two drifts in the same dual-number algebra, so its first- and second-order derivatives propagate automatically.

When AD is disabled (pure-double integration), no transport is required: g is applied to the state as an ordinary function, and the remainder of this section may be skipped.

The construction differs in detail between time-triggered and root-triggered events, because the firing-time perturbation δt enters the dual evaluation differently: directly in the time-triggered case, where $t_e(\theta)$ is supplied by the user and its dual derivatives propagate through the chain rule, and implicitly in the root case, where $t^*(\theta)$ must be reconstructed from the condition $g_e = 0$ via the implicit function theorem.

Time-triggered events

When the firing time is supplied directly as a closed-form expression $t_e(\theta)$, the chain-rule derivatives of t_e with respect to θ are immediately available. The integrator advances on the scalar grid time $t_e^{(0)} := \text{scalar}(t_e)$ and treats the small displacement

$$\delta t = t_e - t_e^{(0)}, \quad \text{scalar}(\delta t) = 0, \quad (15)$$

as a dual number whose value is zero but whose first-order tangents are $\partial t_e / \partial \theta_a$ and whose second-order tangents are $\partial^2 t_e / \partial \theta_a \partial \theta_b$, both inherited automatically from the user-supplied expression for t_e . The state at the grid time $t_e^{(0)}$ is denoted x^- and is itself a dual number that carries the pre-event sensitivities S_θ in its first-order tangent slots.

The transport across the event consists of three steps: a short Heun integration along $\dot{x} = f$ from $t_e^{(0)}$ forward to t_e , the reset map g applied at t_e , and a short Heun integration backward from t_e to $t_e^{(0)}$,

$$\begin{aligned}
\text{forward: } & f_1 = f(x^-, t_e^{(0)}), \quad x_E = x^- + f_1 \delta t, \quad f_2 = f(x_E, t_e), \\
& x^* = x^- + \frac{1}{2}(f_1 + f_2) \delta t, \\
\text{reset: } & x_+^* = g(x^*, t_e), \\
\text{backward: } & \tilde{f}_1 = f(x_+^*, t_e), \quad x_B = x_+^* - \tilde{f}_1 \delta t, \quad \tilde{f}_2 = f(x_B, t_e^{(0)}), \\
& x^+ = x_+^* - \frac{1}{2}(\tilde{f}_1 + \tilde{f}_2) \delta t.
\end{aligned} \tag{16}$$

The forward leg evaluates the right-hand side at both endpoints of the short interval ($t_e^{(0)}$ and t_e); averaging the two values is the standard Heun update and produces a result accurate to second order in δt . The reset map g is then applied on the event surface at t_e , and the backward leg is the same Heun rule shifted from t_e back to the grid time, this time using the post-event right-hand side.

A single Euler step would suffice for first-order sensitivities: since $\text{scalar}(\delta t) = 0$, the powers δt^k for $k \geq 2$ have zero first-order dual content, and any $O(\delta t^2)$ term drops out of $\partial x^+ / \partial \theta$. For second-order sensitivities, however, the curvature term

$$\frac{1}{2}(Jf + \partial f / \partial t) \delta t^2, \quad J = \partial f / \partial x,$$

acquires a non-vanishing second-order dual component, because the square of δt has a non-zero mixed derivative

$$\frac{\partial^2 (\delta t)^2}{\partial \theta_a \partial \theta_b} = 2 \frac{\partial t_e}{\partial \theta_a} \frac{\partial t_e}{\partial \theta_b},$$

despite both its value and its first derivatives being zero. The Heun midpoint absorbs this curvature term through the second right-hand-side evaluation at the shifted state *and* the shifted time. The use of the dual-typed time argument t_e rather than $t_e^{(0)}$ alone in f_2 and $\tilde{f}_2$ is therefore essential: omitting it would discard the $\frac{1}{2}(\partial f / \partial t) \delta t^2$ contribution for systems with explicit time dependence, even though the time shift itself is invisible at the scalar level ($\text{scalar}(\delta t) = 0$).

Root-triggered events

For a root event the firing time $t^*(\theta)$ is defined implicitly by

$$G(t^*(\theta), \theta) := g_e(x(t^*(\theta), \theta), t^*(\theta)) = 0. \tag{17}$$

Three steps lead from this condition to the corrected state at the next grid time: localisation of t^* on the scalar trajectory, reconstruction of the dual residual δt^* via the implicit function theorem, and the same forward-reset-backward Heun transport as in the time-triggered case.

Localisation The integrator reads g_e at the end of every accepted step and at every requested output time. A crossing is accepted on $[t_n, t_{n+1}]$ when the two readings have opposite signs, and also when $g_e(x_{n+1}, t_{n+1})$ is exactly zero, which is what a root falling on a sampled time looks like. A zero at the lower end is not a new crossing: it was reported when that sample was taken. That convention is also what keeps an event from firing a second time on the surface it has just been applied on, where g_e is zero up to round-off.

The crossing is then refined on the dense output of the accepted step, so no extra right-hand-side evaluations are spent on the localisation itself. The iteration is a false-position step whose interpolation factor is clamped to $[0.1, 0.9]$, which bounds the stagnation of the classical rule while keeping its speed on a nearly linear g_e . It stops when the bracket is narrower than the user-supplied root tolerance, after at most fifty iterations, or as soon as an iterate returns $g_e = 0$ exactly. What is returned as $t_*^{(0)} := \text{scalar}(t^*)$ is the end of the final bracket that still carries the sign g_e had before the crossing. The dual residual $\delta t^* = t^* - t_*^{(0)}$ must now be reconstructed from (17), since a localisation on the scalar trajectory provides no information about how t^* depends on θ .

The dual residual via the implicit function theorem The implicit function theorem yields the derivative of $t^*(\theta)$ directly from the constraint $G(t^*, \theta) = 0$, without explicit inversion, provided G is smooth at the crossing and $\partial G/\partial t \neq 0$ there. Differentiating (17) once with respect to θ and applying the chain rule gives

$$\frac{\partial t^*}{\partial \theta} = -\frac{G_\theta}{G_t} = -\frac{\nabla_x g_e \cdot S_\theta}{\nabla_x g_e \cdot f + \partial g_e / \partial t} \Big|_*, \quad (18)$$

in which the denominator is the total time derivative $\dot{g}_e = \nabla_x g_e \cdot f + \partial g_e / \partial t$ along the trajectory at the crossing. The non-vanishing-denominator condition $\dot{g}_e \neq 0$ is the geometric statement that the trajectory crosses the event surface transversally rather than grazing it; for grazing crossings the firing time itself is not differentiable in θ and a separate fallback applies, see the discussion of simultaneous events below. In dual-number form, (18) is realised as the quotient

$$\delta t^* = -\frac{g_e(x^-, t_*^{(0)})}{\dot{g}_e(x^-, t_*^{(0)})}, \quad (19)$$

where both g_e and $\dot{g}_e$ are evaluated on the dual-typed state x^- at the scalar grid time. The codegen-emitted partials $\nabla_x g_e$ and $\partial g_e / \partial t$ are required only for the assembly of $\dot{g}_e$; the dual chain rule propagates $\nabla_x g_e \cdot S_\theta$ through $g_e(x^-, t_*^{(0)})$ automatically. The scalar residual of the numerator (the bisection localisation tolerance) is set to zero before the division so that the value of δt^* is exactly zero by construction; only its dual derivatives remain.

The first-order dual content of (19) reproduces (18) exactly. At second order the dual quotient misses one term, because g_e is evaluated at the fixed scalar time $t_*^{(0)}$ rather than at the dual-typed time t^* : the quotient rule cannot supply the implicit G_{tt} -contribution that would arise from differentiating g_e in its time slot. Expanding (17) to second order in θ and matching coefficients identifies the missing piece. Writing $\delta t^* = \delta t_{(1)}^* + \delta t_{(2)}^* + \dots$ and using $G(t_*^{(0)}, \theta^{(0)}) = 0$,

$$0 = G_t \delta t_{(2)}^* + \frac{1}{2} G_{tt} (\delta t_{(1)}^*)^2 + G_{t\theta} \delta t_{(1)}^* \delta \theta + \frac{1}{2} G_{\theta\theta} \delta \theta^2,$$

of which the dual quotient supplies the $G_{t\theta}$ - and $G_{\theta\theta}$ -contributions but not the G_{tt} -contribution. The corrected update is therefore

$$\delta t^* \leftarrow \delta t^* - \frac{1}{2} \frac{G_{tt}}{G_t} (\delta t^*)^2, \quad (20)$$

where

$$G_{tt} = \frac{d^2}{dt^2} g_e(x(t), t) \Big|_*$$

is the second total time derivative of g_e along the trajectory. cppDE supplies G_{tt} either through a codegen-emitted analytic callback or, when unavailable, through a forward finite difference of $\dot{g}_e$. Equation (20) touches only the second-order dual components of δt^* ; the value and first-order parts are unchanged.

Forward-reset-backward sandwich With δt^* in hand, the same forward-reset-backward Heun procedure of (16) carries x^- across the event to the grid time $t_*^{(0)}$, with δt replaced by δt^* and the event time t_e replaced by $t_*^{(0)} + \delta t^*$. The five right-hand-side evaluations and the dual-typed time arguments play exactly the same role as in the time-triggered case.

Simultaneous events Two or more root conditions can cross zero within the same accepted step. cppDE localises each crossing independently and groups any that share the same $t_*^{(0)}$ to within the localisation tolerance into a simultaneous-event cluster. For such a cluster the forward Heun shift is performed once to the common event surface, every triggered reset is applied at that surface (the reset maps are assumed to commute), and a single backward Heun shift returns the state to the grid time. The cost is four right-hand-side evaluations independent of the number of simultaneous events. The dual residual δt^* is computed once from the first event in the cluster with non-zero $\dot{g}_e$ at the surface; the remaining events share the same δt^* , since they all sit on the same grid time with the same parameter dependence inherited through x^- .

If every triggered event in the cluster has $\dot{g}_e = 0$ at the surface (a tangential crossing of all event functions), the denominator in (18) vanishes and the implicit function theorem fails to deliver $\partial t^*/\partial\theta$: the firing time is then insensitive to θ at first order. cppDE detects this case by the smallness of $|\dot{g}_e|$ and falls back to applying the reset as a plain map without the saltation transport, which is mathematically correct in this limit because the two saltation contributions $f^\pm(\partial t^*/\partial\theta)$ both vanish.

Conditions switched on by a jump A reset is a jump, not a crossing of the continuous solution. A root condition that a reset makes true is therefore invisible to the sign-change search above, and cppDE reads every condition on both sides of a jump instead. A condition whose sign changes across the jump fires there, one per sweep so that a reset can switch on the next condition, and one that already held before the jump does not fire again.

The switched resets are applied on the surface of the jump, between the forward and the backward shift of the event that caused it, rather than after the transport has returned to the grid time. The composition of the two reset maps is then carried across the discontinuity by a single sandwich, and its sensitivities are those of the same reset written as a time-triggered event at that time, to both orders.

Terminal conditions are excluded from this path. They stop the integration rather than reset a state, which the reset step has no way to report.

Connection to the saltation matrix

The standard treatment of sensitivity transport across a discontinuity yields a closed-form correction, the *saltation matrix*, which generalises the bare reset Jacobian $\partial g/\partial x$ by an additive term accounting for the θ -dependence of the firing time:

$$S_\theta^+ = \frac{\partial g}{\partial x} S_\theta^- + \left(\frac{\partial g}{\partial t} + \frac{\partial g}{\partial x} f^- - f^+ \right) \frac{\partial t^*}{\partial \theta}.$$

The first-order content of the dual-number forward-drift-reset-backward-drift construction reproduces this expression exactly, both for time-triggered events ($\partial t^*/\partial\theta = \partial t_e/\partial\theta$ supplied directly through the user expression for $t_e(\theta)$) and for root events ($\partial t^*/\partial\theta$ computed from (18)). The motivation for adopting the implicit-function and dual-number formulation in cppDE is that it extends naturally to second-order sensitivities, for which the classical saltation-matrix form has no closed-form analogue in the literature. The implementation resides in `cppde_integrate_times.hpp`.

2.9 Reverse-mode sensitivities: the discrete adjoint

Forward sensitivities cost one additional state vector per direction in θ , so a gradient of a scalar objective over M parameters costs M times the value alone. When the quantity of interest is a single scalar J , a likelihood or a residual sum of squares, the transpose of that computation delivers the whole gradient at a cost independent of M . cppDE emits this second differentiation mode under `cppODE(..., sweep = "reverse")`.

The quantity computed is a directional contraction rather than the sensitivity matrix. Given a cotangent $W \in \mathbb{R}^{n_t \times n_x}$ attached to the observed trajectory, the reverse mode returns

$$\bar{\theta}_j = \sum_{o=1}^{n_t} \sum_{i=1}^{n_x} W_{oi} \frac{\partial x_i(t_o)}{\partial \theta_j}, \quad j = 1, \dots, n_x + n_p, \quad (21)$$

in one sweep. For a least-squares objective W is the residual weighted by the error model, and $\bar{\theta}$ is $\nabla_\theta J$ exactly. Several cotangents may be supplied as columns of a three-dimensional array, one sweep each.

The codual scalar and its tape

Reverse-mode AD cannot be realised by an augmented arithmetic alone. A dual number carries its derivative forward through each operation, so the derivative is available the moment the value is; a reverse derivative

is a sum over all downstream uses of a quantity, and none of them has happened yet when the operation executes. The computation must therefore be recorded and traversed backwards.

The recording type is `cppde::codual<T>`. It holds a value and an index into a shared tape; every overloaded operation appends one node,

$$\text{node} = (a, b, \partial c / \partial a, \partial c / \partial b),$$

naming the tape slots of its operands and the local partials of the result with respect to them. The tape is thus a linearisation of the straight-line program that produced the value. The reverse sweep walks it from the last node to the first, propagating adjoints by

$$\bar{a} += \frac{\partial c}{\partial a} \bar{c}, \quad \bar{b} += \frac{\partial c}{\partial b} \bar{c},$$

which is the chain rule read right to left. Because the partials are stored as T and the adjoint accumulator is a `std::vector<T>`, the construction is generic in the scalar type; the shipped instantiation is $T = \text{double}$.

Checkpoint per step and replay

Taping an entire integration is not affordable. A single evaluation of f on a model of moderate size leaves hundreds of nodes, and an adaptive run takes hundreds to thousands of steps; the tape would be proportional to their product, and the reverse mode would trade a factor M in time for a comparable factor in memory.

cppDE avoids this with a checkpoint-and-replay scheme. The forward pass runs in plain `double`, no AD type at all, and records, for each accepted step, the state it started from, its start time, and the size it took. That checkpoint store grows as $n_{\text{steps}} \times (n_x + 2)$ doubles, which is the memory of the trajectory itself and not of its derivative.

The reverse pass then walks the steps newest first. For each one it rewinds the tape, re-executes that one step under `codual<double>` starting from its checkpoint, seeds the cotangent the later step handed back together with any observation falling inside it, and sweeps. The peak tape is therefore one step wide, independent of the length of the run, and after the first step the tape allocates nothing further: rewinding keeps the capacity. Parameters are shared by every step, so their adjoint accumulates across the loop, which is the discrete form of the quadrature that the continuous adjoint writes as an integral.

Observations do not generally fall at step ends. The dense-output interpolant of Section 2 is recorded on the tape of the step whose bracket contains the observation time, so a single sweep carries both the step and every interpolation inside it, and the cotangent reaches the step's state through the same polynomial the forward run evaluated.

The corrector without replaying its iteration

For BDF/NDF and Rosenbrock the step is not an explicit formula. A naive replay would re-run the Newton iteration under `codual` and tape every iterate, which both inflates the tape and differentiates the iteration rather than the equation it solves.

cppDE instead differentiates the implicit function directly. Let the accepted step satisfy $F(x_n, x_{n-1}, \theta) = 0$, with F the corrector residual and $\partial F / \partial x_n$ the iteration matrix the forward run already factorised. The implicit function theorem gives

$$\bar{u} = \left(\frac{\partial F}{\partial x_n} \right)^{-\top} \bar{x}_n, \quad \overline{(\cdot)} += - \left(\frac{\partial F}{\partial (\cdot)} \right)^{\top} \bar{u},$$

so one transposed linear solve replaces the entire iteration history. The factorisation is the same LU or KLU object the forward step built, reused transposed; the tape holds only the residual, not the iterates. The linear stage solves of Rosenbrock are handled the same way, one transposed solve per stage.

The reverse mode is more exacting of the linear algebra than the forward mode is. A Newton corrector iterates, so a factorisation that has become numerically poor is corrected by the next iterate and never observed; the adjoint solve uses its result once and directly. This is why the sparse path checks the reciprocal pivot growth of a reused KLU ordering rather than trusting `klu_refactor` to report failure.

Events backwards

An intervention between two steps is replayed as the composition it was: the drift to the root, the reset map, and the restart. Which events fired and where the root sat are control decisions and are read back from the record, not decided again; what is re-executed is the arithmetic. The saltation construction of the previous section is traversed in reverse order, so the transposed saltation matrix carries the cotangent back across the jump, and $\partial t^*/\partial \theta$ from (18) enters through the same expression, transposed. For a multistep method the restart rebuilds the whole Nordsieck history out of the post-jump state, so the cotangent of that history collapses onto a cotangent of the single state, which is what makes an event boundary cheap in reverse.

The step grid is not differentiated

The adaptive controller chooses h_n from the error estimate, which depends on θ ; the computed trajectory therefore depends on θ both through the differential equation and through the grid. The reverse mode holds the grid fixed at what the forward run chose: each step is replayed at the size read from its checkpoint, and that size is a constant on the tape.

This is Bock's principle of internal numerical differentiation. The nominal integration adapts freely; the adaptive decisions are then regarded as *specified*, and the derivative is taken of the scheme that those decisions produced. The alternative, taping the control law so that h_n becomes a differentiable function of the error and hence of θ , was implemented in cppDE behind a switch and removed. Two reasons decided it. First, $h(\theta)$ is piecewise smooth with jumps, not smooth: the step count changes discontinuously as θ varies, and a sweep over θ shows the resulting difference quotient scaling with the sweep density, which is the signature of a jump. Automatic differentiation sees only the smooth part, so taping the controller yields the true gradient plus a smooth grid term minus the jump term that dominates it, a third quantity, and not the derivative of anything the program reports. Second, the recorded control law includes the rejected attempts, which is precisely the tape growth the checkpoint scheme exists to avoid.

Relation to the forward mode

Both modes are exact derivatives, but of two different discretisations. The forward mode's error norm (14) maximises over the sensitivity columns, so a run with sensitivities active takes a finer grid than the same model solved for its value alone. The reverse mode's forward pass carries no tangents, so its grid is the value-only grid.

The consequence is a consistency property that is worth stating explicitly. Under `sweep = "reverse"` the returned value and the returned gradient belong to the same trajectory: the gradient is the exact derivative of the number that was reported. Under forward sensitivities they do not, the value being reported from a grid that the tangents made finer. The two gradients agree to $\mathcal{O}(\text{tol})$ and the difference falls with the tolerance, so the distinction is invisible at tight tolerance and matters where a trust-region step compares a predicted reduction with an achieved one.

The reverse mode carries no `sens1` and no `sens2`; second derivatives remain a forward-mode facility. It supports all four time-stepping methods, time- and root-triggered events, forcings, the batch entry point and the sparse linear solver. The implementation resides in `cppde_codual.hpp`, `cppde_codual_tape.hpp`, `cppde_reverse_step.hpp` and `cppde_reverse_trajectory.hpp`.

2.10 Method selection

The recommended choice of method for the most common scenarios is summarised in Table 4.

The cost of an inappropriate choice is asymmetric. An explicit method applied to a stiff problem will fail or be driven to step sizes that make the integration uneconomic, while an implicit method applied to a non-stiff problem incurs only a constant overhead per step. In the absence of prior knowledge of the spectrum of J , BDF/NDF is the safer default.

Situation	Recommended method
Default; stiffness unknown	"bdf"
Stiff, frequent events	"rb4"
Stiff, smooth, large n_x with sparse Jacobian	"bdf" with <code>sparse = TRUE</code>
Non-stiff, expensive right-hand side	"tsit5"
Non-stiff, cheap right-hand side, long horizon, high accuracy	"adams"

Table 4: Recommended `method` argument for common integration scenarios. Stiffness is the dominant factor; in its absence, right-hand-side cost and required accuracy decide.

3 `cvode()`

The `cvode()` entry point compiles a model against the system-installed SUNDIALS CVODE/CVODES library and exposes the result through the same `solveODE()` interface as `cppODE()`. The two backends therefore differ only in the numerical core; the codegen pipeline, the `events`, `forcings`, `rootfunc`, and `fixed` arguments, the shape of the returned object, and the column ordering of `parms` and `senslni` all behave identically. SUNDIALS (≥ 6.0) must be installed on the build host; KLU (used for sparse Jacobians) is detected the same way and is required when `sparse = TRUE`.

3.1 What is shared

The symbolic Jacobian $J = \partial f / \partial x$ is derived through SymPy at code-generation time and emitted as analytic C++ exactly as in `cppODE()`. The KLU sparse linear solver path activates automatically when the Jacobian is sufficiently sparse and the SuiteSparse libraries were detected at install time; the dense fallback uses LAPACK. Forcing functions are PCHIP-interpolated, events apply a discontinuous reset to the state, and root events fire when a user-supplied scalar function crosses zero.

3.2 What differs

Sensitivity right-hand side. CVODES integrates the analytic forward-sensitivity equation

$$\dot{S}_k = J(x, p) S_k + \frac{\partial f}{\partial p_k}, \quad k = 1, \dots, n_p, \quad (22)$$

directly. The in-tree backend recovers the same equation by evaluating f on dual-typed state, cf. (11). The two formulations are mathematically equivalent. Numerically, the analytic right-hand side avoids the per-evaluation arena allocation required by the dual-number path and is preferable on large problems with a sparse $\partial f / \partial p$; conversely, the dual-number path benefits from SymPy's common-subexpression elimination on small problems for which integrator control flow dominates the cost.

Available methods. CVODE provides only the multistep families. Calling `cvode(method = "bdf")` or `cvode(method = "adams")` selects between them; the single-step methods `rb4` and `tsit5` are not available through this backend. The cost-per-step accounting and the stability properties documented for the multistep methods in Section 2 therefore carry over directly.

Second-order sensitivities. CVODES exposes only first-order forward sensitivities; the nested-dual construction has no counterpart in the SUNDIALS API. `cvode()` therefore does not accept `deriv2 = TRUE`.

Saltation transport across events. Events on `cvode()` models use the same forward-drift-reset-backward-drift sequence as `cppODE()`, with the first-order saltation correction of equations (16) and (18). The transport is performed in the generated glue layer rather than inside CVODES, since CVODES exposes no entry point at which the sensitivity solver can be informed of the discontinuity.

Reparametrisation seeding. The chain-rule construction $S_\theta = S_p \partial p / \partial \theta$ of equation (12) is identical to the `cppODE()` case. `solveODE()` accepts the same `senslni` matrix shapes (legacy, full, and partial) and the same row-name conventions; only second-order S_θ is unavailable.

4 cppFUN()

The `cppFUN()` entry point compiles a system of algebraic expressions

$$y_i = g_i(x_1, \dots, x_{n_x}, p_1, \dots, p_{n_p}), \quad i = 1, \dots, n_y, \quad (23)$$

together with optional first- and second-order derivatives. No time integration is performed: each call evaluates g at one or more rows of (x, p) values and returns the corresponding outputs, Jacobians, and Hessians as requested. Two use cases motivate the function. The first is the evaluation of observation maps $y = g(x, p)$ that connect integrator output to data in likelihood-based inference; the mapping is applied at every observation point and its Jacobian enters the gradient of the log-likelihood. The second is the evaluation of reparametrisation Jacobians $\partial p / \partial \theta$ that supply the seed matrix in equation (12) when the dynamic model is fitted in a transformed parameter space.

4.1 Unified API and two computational paths

Both values of the `derivMode` argument expose the same R-side methods: `func`, `jac`, `hess`, and the combined `evaluate`. They differ only in how the underlying C entry points are generated and dispatched.

- **"symbolic"**: the analytic Jacobian and Hessian of g are derived by SymPy at code-generation time. The compiled module exposes `_eval`, `_jacobian` (when `deriv`), and `_hessian` (when `deriv2`). When seeds are supplied, the chain rule is applied by BLAS-3 DGEMM through two extern "C" wrappers `_chain_jac` and `_chain_hess`, which delegate to the inline routines in `cppde_chain_blas.hpp`.
- **"forward"** (default): no SymPy step. The compiled module exposes `_eval`, `_eval_ad` (when `deriv`, on `cppde::dual<double, 0>`), and `_eval_ad2` (when `deriv2`, on the nested-dual alias `cppde::dual2nd<double, 0>`). The chain rule is applied inside the AD evaluation by seeding the dual-number tangent slots; the same source that evaluates g over $\mathbb{R}$ then also evaluates the chain-ruled $\partial y / \partial \theta$ and $\partial^2 y / \partial \theta^2$.
- **"reverse"** (default alongside **"forward"**): the vector-Jacobian product `_vjp`, which contracts a cotangent of the outputs into cotangents of the variables and parameters at a cost independent of the upstream parameter count. It instantiates the expression body a second time, over `cppde::codual`, so it is separately requestable: a caller that only ever multiplies by the Jacobian should not compile it.

"symbolic" is a backend for the forward Jacobian rather than a direction of its own, and cannot be combined with the other two; there is no symbolic vector-Jacobian product. **"forward"** and **"reverse"** are independent build products and may be asked for together, which is the default.

The two paths are equivalent in exact arithmetic. The forward AD path avoids the SymPy expansion of $\partial g / \partial p$ and $\partial^2 g / \partial p^2$, whose intermediate expressions can grow sharply with the depth of the compositions in g . The symbolic path benefits from SymPy's common-subexpression elimination when the analytic derivatives remain compact. The reverse entry is described in Section 2 for the integrator; here it is the same construction without the time stepping, and one sweep over a straight-line program.

Call	dual path	symbolic path
<code>func(x, p)</code>	<code>_eval</code>	<code>_eval</code>
<code>jac(x, p)</code> (no seeds)	<code>_eval_ad</code> with identity seed	<code>_jacobian</code>
<code>jac(x, p, dX, dP)</code>	<code>_eval_ad</code> with user seeds	<code>_jacobian + _chain_jac</code>
<code>hess(x, p)</code> (no seeds)	<code>_eval_ad2</code> with identity seed	<code>_hessian</code>
<code>hess(x, p, dX, dP[, dX2, dP2])</code>	<code>_eval_ad2</code> with all seeds	<code>_hessian + _chain_hess</code>
<code>evaluate(..., deriv2 = FALSE)</code>	one <code>_eval_ad</code> -pass	<code>_eval + _jacobian (+ chain)</code>
<code>evaluate(..., deriv2 = TRUE)</code>	one <code>_eval_ad2</code> -pass	<code>_eval + _jacobian + _hessian (+ chain)</code>

Table 5: Dispatch within `cppFUN()`. Both modes accept the same arguments and return arrays of the same shape; only the C entries called and the location of the chain-rule contraction differ.

The combined entry point `evaluate()` is provided because in the dual path a single nested-dual evaluation produces the value, the first-order sensitivities, and the second-order sensitivities together; obtaining all three through separate calls to `func`, `jac`, and `hess` would repeat the underlying evaluation three times. In the symbolic path `evaluate()` simply combines the three independent C calls and applies the chain rule once.

4.2 Chain rule via dX, dP, dX2, dP2

The seed arguments encode the Jacobian and (optionally) Hessian of an upstream parameter map $\Phi : \theta \mapsto (x, p)$:

- `dX[obs, name, theta]` – $\partial x_i / \partial \theta_a$, indexed per observation row, per state name, per direction θ_a .
- `dP[name, theta]` – $\partial p_j / \partial \theta_a$, constant across observations.
- `dX2[obs, name, theta1, theta2]` – second-order sensitivities $\partial^2 x_i / \partial \theta_a \partial \theta_b$, optional. Defaults to zero (linear / affine Φ).
- `dP2[name, theta1, theta2]` – second-order parameter sensitivities $\partial^2 p_j / \partial \theta_a \partial \theta_b$, optional. Defaults to zero.

Symbols listed in `fixed` have their seed rows zeroed before the contraction; symbols absent from the seeds also contribute zero, not an error.

In the symbolic path the seeds are bundled into one matrix $S \in \mathbb{R}^{n_{\text{obs}} \times n_{\text{diff}} \times n_{\theta}}$ over the canonical-symbol basis $n_{\text{diff}} = n_x + n_p$. The chain rule for the Jacobian is

$$\left. \frac{\partial y}{\partial \theta} \right|_{\text{obs}} = J(x_{\text{obs}}, p) \cdot S_{\text{obs}}, \quad (24)$$

implemented as one DGEMM per observation in `cppde::chain_jac`. For the Hessian, two contributions combine,

$$\left. \frac{\partial^2 y}{\partial \theta \partial \theta^\top} \right|_{\text{obs}, o} = S_{\text{obs}}^\top H_o(x_{\text{obs}}, p) S_{\text{obs}} + \sum_i J_{o,i}(x_{\text{obs}}, p) S_{2_{\text{obs}, i, \cdot, \cdot}}, \quad (25)$$

implemented as two DGEMMs plus one DGEMV per (obs, output) in `cppde::chain_hess`. The second term vanishes when Φ is affine (`dX2` and `dP2` unset).

In the dual path the same contractions are obtained automatically by seeding the dual-number tangent slots. Each input scalar x_i becomes

$$\tilde{x}_i = (x_i + dX_{i,\cdot} \varepsilon_{\text{in}}) + (dX_{i,\cdot} + dX2_{i,\cdot,\cdot} \varepsilon_{\text{in}}) \varepsilon_{\text{out}},$$

i.e. a nested dual whose inner tangent slots carry $\partial x_i / \partial \theta$ and whose outer tangent slot k is itself an inner dual carrying $\partial x_i / \partial \theta_k$ as its value and $\partial^2 x_i / \partial \theta_k \partial \theta$ as its inner tangents. The dual-number multiplication rule then propagates both the $S^\top H S$ and the $J \cdot S2$ terms in one nested-arithmetic evaluation; the inner-tangent values of $\tilde{y}$ are read out as $\partial y / \partial \theta$, and the cross slots $\tilde{y}[k][m]$ as $\partial^2 y / \partial \theta_k \partial \theta_m$.

4.3 Pass-through of unmodelled inputs

The `attach.input` flag controls whether unmodelled columns of the input are appended to the output. When `attach.input = TRUE`, columns of the input matrix that do not appear among the named variables of (23) are passed through to the output unchanged: their value rows are copied, their Jacobian rows are zero for variables and the identity for parameters, and their Hessian rows are zero. The convention removes the need to extend g with no-op identities when the observation map does not consume every column of the integrator output, and matches the row-name-driven partial-shape `senslini` convention used by `solveODE()`.

5 Benchmarks

Both backends solve the same problem from the same symbolic input. Right-hand side, analytic Jacobian, events, linear-solver selection, and tolerances are shared, so a head-to-head measurement isolates the numerical core rather than the model formulation.

Setup. The suite comprises 41 problems: 31 published systems-biology models from a PETab benchmark collection and 10 classical test problems (van der Pol at $\mu = 1000$, Robertson, OREGO, HIRES, E5, Pollution, and method-of-lines semidiscretisations of the one-dimensional Brusselator and FitzHugh–Nagumo systems at $N = 24$ and $N = 64$). The state dimension ranges from $n_x = 2$ to 128, the number of differentiated parameters from $M = 1$ to 64. One further model of the collection, Lang_PLOSCoMputBiol2024 at $n_x = 124$, is left out: its solves alone take longer than the rest of the suite together, and it belongs to none of the regimes the comparison turns on. Both backends run their BDF/NDF method with the automatically selected linear solver at $(\text{rtol}, \text{atol}) \in \{(10^{-4}, 10^{-6}), (10^{-7}, 10^{-9}), (10^{-10}, 10^{-12})\}$, without and with first-order sensitivities. Each cell repeats the solve until one batch exceeds a wall-clock floor and reports the median of five such batches, on an otherwise idle single core. Figure 1 shows the resulting ratio $r = t_{\text{cvsode}}/t_{\text{cppode}}$ per problem, aggregated over the three tolerances by geometric mean.

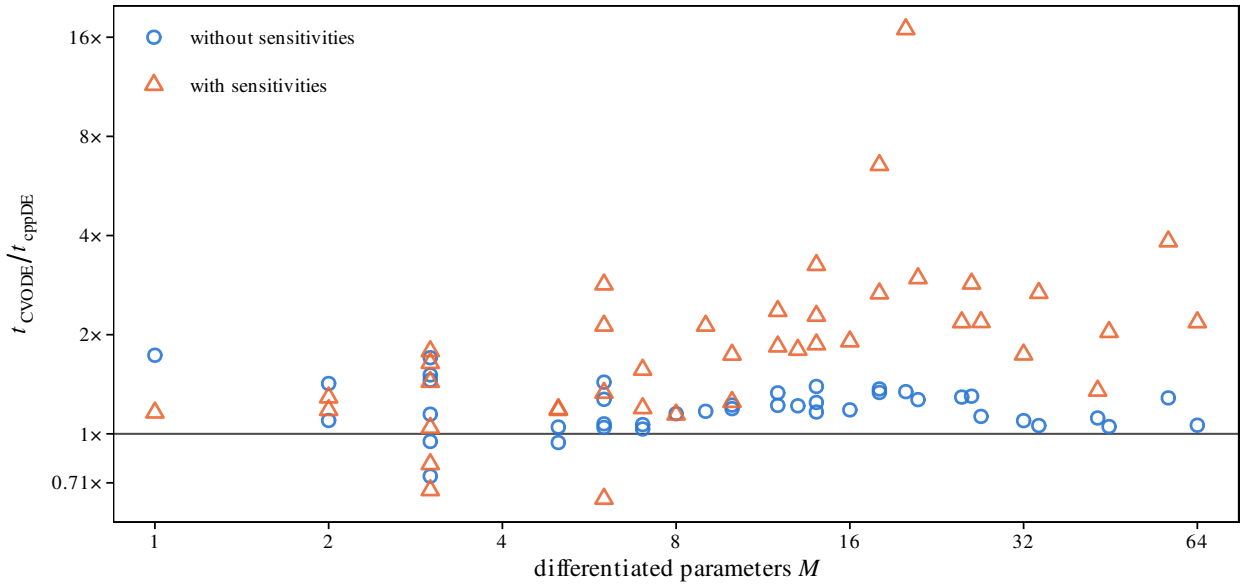


Figure 1: Head-to-head ratio of the two backends, one marker per problem and mode; above the line `cppODE()` is faster. The abscissa is the number of parameters the model differentiates, which the plain solve does not use. Its markers are placed at the same abscissa as their sensitivity counterparts, and their flat course shows that the M dependence belongs to the sensitivity mode alone.

Without sensitivities the in-tree backend is faster on 38 of the 41 problems, by a geometric mean of 1.20, and the ratio is flat in M . The two backends take the same steps: the geometric means of the accepted-step and right-hand-side evaluation ratios are 1.01 and 1.05, which rules out a margin bought by integrating more loosely. This is expected, since the controller, the order selection, and the error constants of the in-tree multistep are those of CVODE, and the margin is therefore a cost-per-step effect. It arises from model specialisation: the right-hand side and the Jacobian are emitted as straight-line C++ and instantiated into the stepper as a functor template, so the calls inline, whereas CVODES reaches them through a function pointer with a `void*` user-data argument; the in-tree state vectors are contiguous with the extent known at the call site, whereas every SUNDIALS vector operation is dispatched through the operation table of the `N_Vector` it acts on. Part of the margin belongs to the code generator rather than to the integrator: common-subexpression elimination is applied to the in-tree right-hand side and Jacobian but not to the CVODES glue layer.

With sensitivities the geometric mean rises to 1.84, and Figure 1 shows that it does so as a function of M : from 1.17 over the nine problems with $M \leq 3$ to 3.26 over the nine with $13 \leq M \leq 24$. CVODES is driven in staggered mode with an analytic sensitivity right-hand side that is invoked once per differentiated parameter, each invocation forming one column of (12) and with it every structurally nonzero entry of J ; since the M invocations are separate calls, those entries are recomputed M times per corrector iteration. The in-tree backend obtains the same M columns from a single evaluation of f on a dual-typed state carrying all M tangents, and forms the shared subexpressions of J once. It reaches the same trajectories with roughly one twelfth of the right-hand-side evaluations.

Where the ordering reverses. The dual evaluation carries a cost per state component that is independent of M : the arithmetic on the value part, the tangent-slab bookkeeping, and the loss of the vectorisation the scalar loop admits. The in-tree sensitivity work therefore scales as $n_x(a + bM)$ with $a > 0$, whereas the CVODES work scales as $M \cdot \text{nnz}(J)$. For a method-of-lines discretisation $\text{nnz}(J) = \mathcal{O}(n_x)$, the state dimension cancels, and the comparison reduces to $a + bM$ against $\mathcal{O}(M)$: small M favours CVODES, and the more so the larger n_x is, since a is paid per component and amortised only over M . Every problem that falls below the parity line of Figure 1 with sensitivities is of this shape, and all of them lie at $M \leq 6$: SalazarCavazos_MBoC2020 ($n_x = 75$, $M = 6$) at 0.64 and the Brusselator at $N = 64$ and $N = 24$ ($M = 3$) at 0.67 and 0.81. Three problems fall below it without sensitivities, two of them by less than 10%.

Linear algebra and accuracy. Fifteen of the models were additionally solved with the linear solver pinned to dense and to sparse. The gain of the sparse path grows with the state dimension in the same way for both backends, from 1.08 at $n_x = 8$ to 2.78 at $n_x = 128$, and the auto-detection selected the faster path in every swept case, so Figure 1 is a comparison at matched linear algebra. Every solve was additionally scored against a reference trajectory at $\text{atol} = 10^{-14}$, $\text{rtol} = 10^{-12}$. Per problem, the deviation of the in-tree backend is a median 1.06 times that of CVODE, with the central 90% of the problems between 0.51 and 1.35. The two are not on equal footing there: the reference is itself a CVODE solution, so CVODE is measured against a tighter run of the same method through the same code, whose truncation error is structurally its own, while the in-tree backend is measured across implementations. The bias runs in CVODE’s favour, which makes the quoted ratio an upper bound. Four of the 852 cells failed and are excluded from the aggregates, all of them the in-tree backend with sensitivities at $\text{rtol} = 10^{-10}$: Oliveira_NatCommun2021 on each of its three linear-solver settings, and Weber_BMC2015. Repeating the suite on eight workers of the same machine, which inflates the absolute times through shared cache and reduced clocks but leaves the ratios intact, reproduces the two aggregates as 1.22 and 1.91, and the per-problem values with a correlation of 0.96 in $\log r$ with sensitivities. Without sensitivities the correlation drops to 0.52, since a substantial fraction of the plain solves complete in well under a millisecond and the measurement is then dominated by the contention the extra workers create. The problems that disagree beyond that scatter are all of that kind, HIRES at 1.42 against 0.77 the widest among them.

References

- [1] A. C. Hindmarsh *et al.*, “SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 31, no. 3, pp. 363–396, 2005, doi: 10.1145/1089014.1089020.
- [2] L. F. Shampine and M. W. Reichelt, “The MATLAB ODE suite,” *SIAM Journal on Scientific Computing*, vol. 18, no. 1, pp. 1–22, 1997, doi: 10.1137/S1064827594276424.
- [3] E. Hairer and G. Wanner, *Solving ordinary differential equations II: Stiff and differential-algebraic problems*. in Springer series in computational mathematics. Springer Berlin Heidelberg, 2010. Available: <https://books.google.de/books?id=zZSv3acps1QC>
- [4] Ch. Tsitouras, “Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption,” *Computers & Mathematics with Applications*, vol. 62, no. 2, pp. 770–775, 2011, doi: 10.1016/j.camwa.2011.06.002.
- [5] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving ordinary differential equations I: Nonstiff problems*. in Springer series in computational mathematics. Springer Berlin Heidelberg, 2008. Available: <https://books.google.de/books?id=F93u7VcSRyYC>

A BDF and NDF coefficients

Source. [2], Table 1; cppDE implementation in `cppde_multistepper.hpp`.

The Numerical Differentiation Formula (NDF) of order q modifies the classical BDF q corrector by adding a term proportional to the Nordsieck predictor error,

$$\sum_{m=1}^q \frac{1}{m} \nabla^m x_n - h_n f(x_n, t_n) - \kappa_q \gamma_q (x_n - x_n^{\text{pred}}) = 0, \quad \gamma_q = \sum_{m=1}^q \frac{1}{m}.$$

Setting $\kappa_q = 0$ recovers BDF q . The values κ_q adopted by cppDE (selectable at compile time via the argument `useNDF`, default `TRUE`) are tabulated in Table 6, together with the empirical step-size gain over classical BDF q and the resulting stability angle.

q	κ_q	Step-size gain over BDF q	Stability angle
1	-0.1850	+26 %	A-stable
2	-0.1111	+26 %	A-stable
3	-0.0823	+26 %	80°
4	-0.0415	+12 %	66°
5	0.0000	0 %	51° (= BDF5)

Table 6: NDF correction coefficients κ_q adopted by cppDE, the empirical step-size gain over classical BDF q at matched local error, and the resulting stability angle.

NDF5 is set to $\kappa_5 = 0$ because the BDF5 stability angle of 51° leaves no headroom for further reduction; NDF5 is therefore identical to BDF5.

B Adams-Moulton coefficients

Source. [1], ℓ -vector formulation as in CVODE; cppDE implementation in `adams_set_coefficients()` inside `cppde_multistepper.hpp`.

In contrast to BDF, the Adams-Moulton coefficients depend on the recent step-size history and are recomputed every step from the Nordsieck array. Define $\xi_j = (t_n - t_{n-j})/h_n$; the ℓ -vector recurrence reads

$$\ell_0(q) = 1, \quad \ell_j(q) = \ell_{j-1}(q) + \xi_j^{-1}, \quad j = 1, \dots, q,$$

followed by a polynomial expansion that yields the corrector weights $\beta_0, \dots, \beta_q$ used in the current step. Because the coefficients are step-size adaptive, no static table can reproduce them in general; the canonical fixed-step values for $h_n = h_{n-1} = \dots$ are given in standard references [5], Table III.1.1 and reduce to the recurrence above in that limit.

The maximum order in cppDE is $q_{\max} = 12$, matching CVODE's default; the order controller may select any $q \in \{1, \dots, 12\}$ at run time.

C Rosenbrock4 coefficients

Source. [3, Sec. IV.7]; identical to the default coefficients in the `rosenbrock4` stepper of Boost.Numeric.Odeint. cppDE implementation in `default_rosenbrock_coefficients` in `cppde_rosenbrock4.hpp`.

A Rosenbrock method of s stages with diagonal coefficient γ is defined, with iteration matrix $W = (\gamma h_n)^{-1} I -$

$J(x_n, t_n)$, by the stage equations

$$W k_i = f\left(x_n + \sum_{j<i} \alpha_{ij} k_j, t_n + c_i h_n\right) + \frac{1}{h_n} \sum_{j<i} \frac{\gamma_{ij}}{\gamma} k_j + h_n d_i \frac{\partial f}{\partial t},$$

$$x_{n+1} = x_n + \sum_{i=1}^s m_i k_i, \quad \hat{x}_{n+1} = x_n + \sum_{i=1}^s \hat{m}_i k_i.$$

The cppDE parameterisation uses $s = 6$ stages with $\gamma = 1/4$. The numerical values of all coefficients are reproduced in the following tables, indexed exactly as in the source code: the diagonal coefficient and the time partials and node positions in Table 7, the stage couplings α_{ij} and γ_{ij} in Tables 8 and 9, the sixth-row weights in Table 10, and the dense-output coefficients in Table 11.

C.1 Diagonal coefficient, time partials, and node positions

γ	0.25	d_1	0.25	c_2	0.386
d_2	-0.1043	d_3	0.1035	c_3	0.21
d_4	0.0362			c_4	0.63

Table 7: Rosenbrock4 diagonal coefficient γ , time-partial coefficients d_i , and stage node positions c_i .

C.2 Stage couplings α_{ij}

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$\alpha_{2,j}$	1.5440000000			
$\alpha_{3,j}$	0.9466785281	0.2557011699		
$\alpha_{4,j}$	3.3148251871	2.8961240160	0.9986419140	
$\alpha_{5,j}$	1.2212245092	6.0191344813	12.5370833293	-0.6878860361

Table 8: Rosenbrock4 stage-coupling coefficients α_{ij} . Row index i is the stage being formed; column index j ranges over earlier stages.

C.3 Stage couplings γ_{ij}

	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$\gamma_{2,j}$	-5.6688000000			
$\gamma_{3,j}$	-2.4300933568	-0.2063599157		
$\gamma_{4,j}$	-0.1073529058	-9.5945622510	-20.4702861481	
$\gamma_{5,j}$	7.4964433140	-10.2468043146	-33.9999035282	11.7089089321

Table 9: Rosenbrock4 stage-coupling coefficients γ_{ij} , appearing in the $(1/h_n) \sum_{j<i} (\gamma_{ij}/\gamma) k_j$ term of the stage equation. Row and column conventions match Table 8.

C.4 Solution and error weights

The advanced solution and the embedded error estimate are formed from the stage values $k_1, \dots, k_5$ via

$$x_{n+1} = x_n + \sum_{i=1}^5 \alpha_{6,i} k_i, \quad e_{n+1} = \sum_{i=1}^5 \gamma_{6,i} k_i,$$

with the sixth-row coefficients listed in Table 10.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
$\alpha_{6,i}$	1.2212245092	6.0191344813	12.5370833293	-0.6878860361	1.0
$\gamma_{6,i}$	8.0832467959	-7.9811329881	-31.5215943287	16.3193054312	-6.0588182388

Table 10: Rosenbrock4 sixth-row coefficients. The $\alpha_{6,i}$ values combine the stage values into the advanced solution x_{n+1} ; the $\gamma_{6,i}$ values supply the embedded order-3 error estimate e_{n+1} .

C.5 Dense-output coefficients

Continuous output on $[t_n, t_{n+1}]$ is constructed from the five intermediate stage values together with two auxiliary coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$, listed in Table 11, that combine the stages with the step solution to produce an order-3 interpolant.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
$d_i^{(2)}$	10.1262350834	-7.4879958776	-34.8009186156	-7.9927717076	1.0251377233
$d_i^{(3)}$	-0.6762803393	6.0877146517	16.4308432089	24.7672251142	-6.5943891257

Table 11: Rosenbrock4 dense-output coefficients $d_i^{(2)}$ and $d_i^{(3)}$. Together with the stage values they form the order-3 interpolant on $[t_n, t_{n+1}]$ used to serve user output points falling between accepted steps.

D Tsitouras 5(4) coefficients

Source. [4], Table 1 and equations (7)–(8); cppDE implementation in `cppde_tsit5.hpp`.

The Butcher tableau of the Tsitouras 5(4) pair, reproduced in Table 12, has $s = 7$ stages and obeys the simplifying assumption $a_{i,1} = 0$ for $i \geq 2$; the seventh stage is the FSAL stage. Both rows of weights are listed beneath the A matrix: b_i for the order-5 solution and $\hat{b}_i$ for the embedded order-4 estimate.

0							
0.161	0.161						
0.327	-0.0084806555	0.3354806555					
0.9	2.8971530571	-6.3594484900	4.3622954329				
0.9800255409	5.3258648284	-11.7488835641	7.4955393429	-0.0924950664			
1.0	5.8614554429	-12.9209693178	8.1593678986	-0.0715849733	-0.0282690504		
1.0	0.0964607668	0.01	0.4798896504	1.3790085741	-3.2900695154	2.3247105241	
b_i	0.0964607668	0.01	0.4798896504	1.3790085741	-3.2900695154	2.3247105241	0
$\hat{b}_i$	0.0946807557	0.0091835655	0.4877705284	1.2342975669	-2.7077123499	1.8666284182	0.0151515152

Table 12: Butcher tableau of the Tsitouras 5(4) pair. The first column lists the node positions c_i ; the centre block is the strictly lower-triangular coefficient matrix a_{ij} . The b_i row gives the order-5 weights, and the $\hat{b}_i$ row gives the embedded order-4 weights.

The seventh stage is $k_7 = f(x_{n+1}, t_n + h_n)$, which is re-used as k_1 at step $n + 1$. Because $b_7 = 0$, the order-5 solution does not depend on k_7 ; because $\hat{b}_7 \neq 0$, the embedded order-4 estimate does. The local-error estimator used by the step controller is

$$e_n = h_n \sum_{i=1}^7 (\hat{b}_i - b_i) k_i.$$

The seven differences $\hat{b}_i - b_i$ are stored in `cppde_tsit5.hpp` as the coefficients `e1`, ..., `e7` and reproduced in Table 13.

i	$\hat{b}_i - b_i$
1	-0.0017800111
2	-0.0008164345
3	0.0078808780
4	-0.1447110072
5	0.5823571655
6	-0.4580821059
7	0.0151515152

Table 13: Tsit5 embedded error coefficients $\hat{b}_i - b_i$, stored in `cppde_tsit5.hpp` as `e1`, ..., `e7`. The seven values sum to numerical zero, the consistency condition for the embedded pair.

The seven values sum to numerical zero, as required for the consistency of the embedded pair, and the row $a_{7,j}$ of the A matrix coincides with b_j for $j = 1, \dots, 6$, which is the FSAL property.